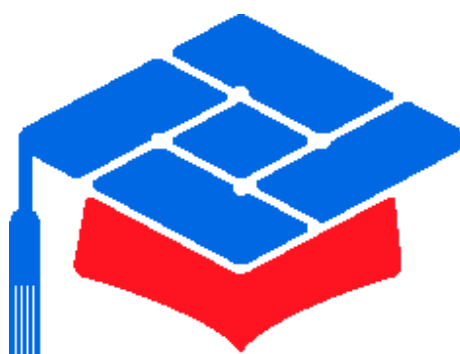




第十届

全国大学生集成电路创新创业大赛

报告类型：技术说明书

参赛杯赛：“七星微”企业命题

作品名称：基于 RISC-V 的五级流水 CPU 设计与 FPGA 验证

队伍编号：CICC1003618

团队名称：芯银河战队

目 录

重要内容预览	1
1 作品概述	2
1.1 设计目标	2
1.2 设计依据与方法	2
1.3 核心设计亮点	2
2 CPU 架构设计	3
2.1 总体结构	3
2.2 五级流水线	3
2.3 指令集与性能优化	4
2.4 控制冒险处理	5
3 Verilog 建模规范	5
3.1 模块边界	5
3.2 编码约定	5
3.3 注释与可追溯性	6
4 验证计划与测试报告	6
4.1 验证策略	6
4.2 关键性能结果	7
4.3 优化前后差异	7
5 FPGA 适配指南	8
5.1 板级配置	8
5.2 资源与时序	8
5.3 上板连接说明	9
6 AI 工具使用声明	9
7 结论	10
参考文献	11
证据索引	12

重要内容预览

表 1 作品核心亮点与实测结果

项目	结果
CPU 架构	RV32I 五级顺序流水，包含 IF、ID、EX、MEM、WB，配套最小 SoC 与 FPGA 顶层。
指令与优化	最终提交路径支持 RV32I、Zmmul、Zba/Zbb/Zbs 与 JAL 早跳转；Zbc、XThead indexed memidx、IDBR 作为参数化探索路径，保留回归测试记录。
CoreMark 结果	4.137461 CoreMark/MHz, 413.746136 iter/s @ 100MHz, 2416941 ticks。
FPGA 结果	Xilinx PYNQ-Z2, CPU 50.0 MHz, 4934 LUT / 2327 FF / 4 BRAM / 15 DSP，满足 LUT < 5000。
时序与上板	实现后 WNS=+0.440 ns、WHS=+0.151 ns，硬件下载日志检出 PROGRAM_OK: xc7z020_1。

本文面向初赛评审，围绕架构设计、RTL 建模、验证结果与 FPGA 适配四条主线展开，重点呈现本作品在高性能、低资源占用与工程可复现性方面的实现亮点。

1 作品概述

1.1 设计目标

YH_rv_cpu 是面向 FPGA 原型验证的轻量级 RISC-V CPU。设计目标是在 $LUT < 5000$ 的资源约束内，完成可综合、可仿真、可上板的五级流水处理器原型，并兼顾性能提升、控制复杂度和低功耗导向实现。为此，设计重点保留对基准程序最敏感的热点路径与热点指令扩展，避免引入面积和时序代价较高但收益有限的复杂结构。

工程实现包括：

- ISA: RV32I + Zmmul + Zba/Zbb/Zbs，并保留 Zbc、XThead、IDBR 参数化探索路径；
- 控制流优化: JAL early redirect 与低面积静态分支处理；
- FPGA: Xilinx PYNQ-Z2，器件 xc7z020clg400-1，CPU 50.0 MHz；
- 性能指标: 4.137461 CoreMark/MHz。

1.2 设计依据与方法

RISC-V 开放 ISA 具有模块化扩展特征，便于处理器在性能、面积和功耗之间按应用场景做取舍 [1, 3]。本设计遵循“基线指令正确、热点扩展增强、控制路径受限前移”的方法：以 RV32I 五级流水作为可验证基线，以 Zmmul 和 Bitmanip 子扩展覆盖 CoreMark、Dhrystone 等整型程序中的乘法、地址生成、位操作与条件选择热点 [2]，同时避免引入会显著推高 LUT 与时序压力的复杂乱序结构。

在微结构层面，五级顺序流水仍是 FPGA 软核 CPU 中兼顾教学可解释性、时序收敛和资源效率的常用结构。相关 RISC-V 软核研究表明，面向分支、前递、访存和乘法路径做局部优化，可在保持轻量实现的同时取得可观性能收益 [4, 5]。本作品据此将优化集中在可量化验证的短路径上：分支与 JAL 的前端重定向、Zmmul 乘法保留、Bitmanip 热点指令保留，以及可关闭的参数化扩展组织。与应用级大核常见的高面积结构相比，单发射顺序流水更适合本赛题 $LUT < 5000$ 的硬约束和 PYNQ-Z2 原型平台 [6]。

1.3 核心设计亮点

本设计围绕轻量级高性能 RISC-V 处理器的常见优化方向展开，重点落在前端控制流缩短、热点扩展保留和 PPA 受约束条件下的选择性增强。

- 热点扩展选择性保留：冻结路径保留 Zmmul 与 Zba/Zbb/Zbs，面向乘法、位操作和地址生成等高频模式提升吞吐；Zbc 与 XThead 子集作为参数化探索路径，用于展示

进一步性能潜力；

- **前端控制流提前化**：在不显著拉长关键路径的前提下，把部分分支比较与 JAL 跳转前移至 ID 阶段，降低控制冒险的恢复开销；
- **低资源低功耗导向**：采用五级顺序流水、同步存储体与参数化扩展开关，移除硬件除法器与高代价控制逻辑，降低面积、切换活动和时序压力；
- **工程闭环完整**：形成从 RTL、TestBench、基准程序、综合实现到上板下载日志的可追溯证据链，便于复现与评审核查。

2 CPU 架构设计

2.1 总体结构

系统采用“CPU Core + 最小 SoC + FPGA 顶层”的三层组织方式。CPU Core 负责指令取指、译码、执行、访存、写回、CSR 和异常处理；SoC 层连接同步指令存储、数据存储与 MMIO；FPGA 顶层完成 PYNQ-Z2 时钟、复位、按键、LED 和 UART 适配。

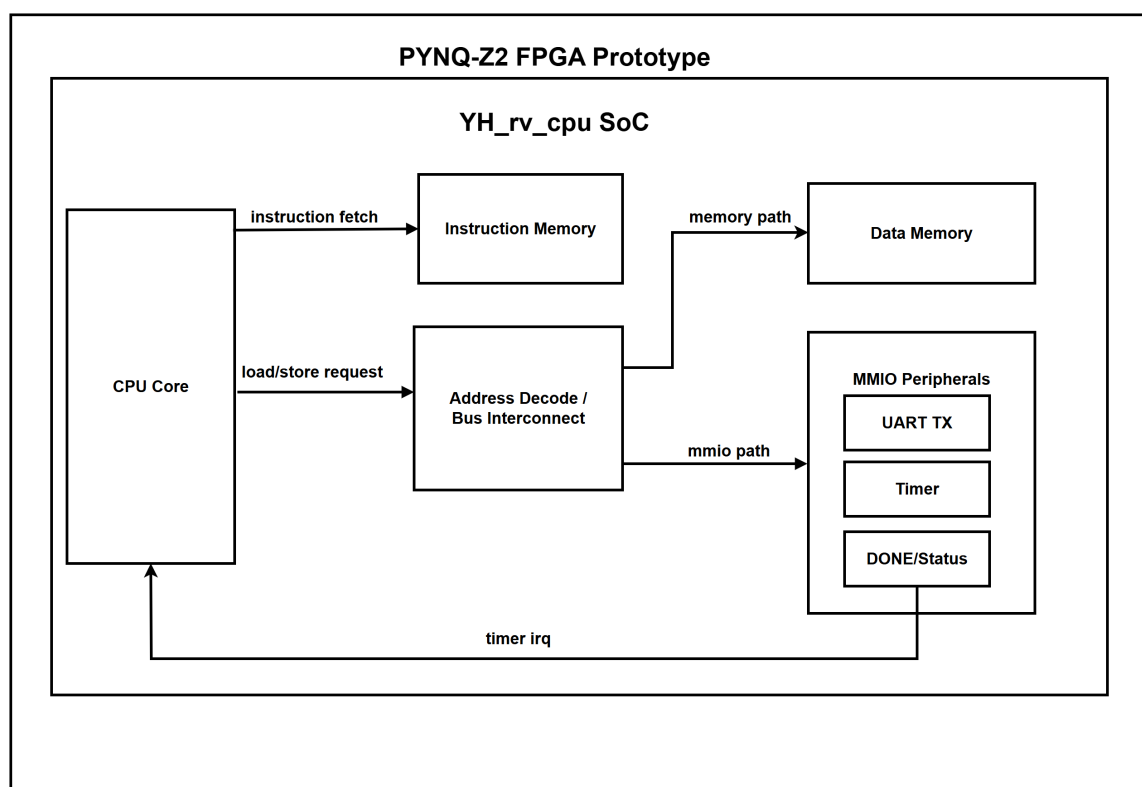


图 1 系统总体架构图

2.2 五级流水线

处理器核心为五级顺序流水结构，阶段划分如下：

表 2 流水线阶段职责

阶段	职责
IF	维护 PC，发起指令存储访问，处理 redirect、预测跳转和复位入口。
ID	译码指令，读取寄存器，生成立即数、控制信号、分支早判定与冒险请求。
EX	执行 ALU、乘法、位操作、分支/JAL/JALR 目标计算和异常条件判断。
MEM	处理 load/store、字节使能、非对齐检查和同步数据存储访问。
WB	选择 ALU/MEM/PC+4/CSR 结果写回寄存器堆。

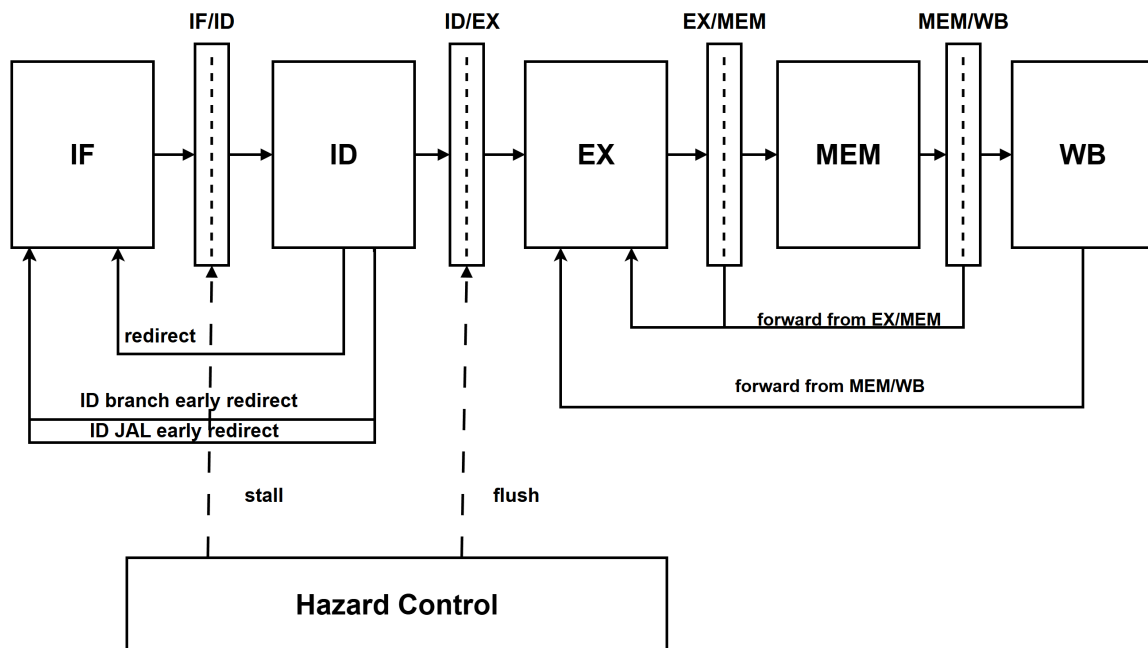


图 2 五级流水与关键控制通路图

2.3 指令集与性能优化

面向 CoreMark、Dhrystone 与通用整型程序热点，处理器重点保留以下低成本高收益优化：

- **Zmmul:** 保留硬件乘法并移除硬件除法路径，使乘法操作吞吐量得益于专用乘法器，同时控制 LUT 与关键路径增长；
- **Zba/Zbb/Zbs:** 增强地址计算、逻辑融合与位提取能力，提升热点位操作与地址生成效率，并在 PYNQ-Z2 上保持 LUT < 5000；
- **Zbc/XThead/IDBR 探索路径:** 相关 RTL 与定向回归测试作为性能探索材料予以保留，但在最终冻结的 bitstream 中，为降低 LUT 资源占用与时序压力而未启用；

- **JAL early redirect:** 对 JAL rd!=x0 在 ID 阶段提前给出目标 PC，并在 EX 阶段完成一致性校验，降低前端重定向开销。

2.4 控制冒险处理

控制冒险统一通过 redirect 机制处理。EX 阶段保留分支、JAL、JALR、trap、mret 的完整重定向能力；ID 阶段仅对跳转目标可提前确定的路径实施重定向。基准测试统计显示，引入 JAL 早跳转后，EX 阶段 JAL redirect 计数降为 0，CoreMark 总完成周期由 1974371 降至 1971888，说明前端重定向代价得到有效压缩。

3 Verilog 建模规范

3.1 模块边界

工程采用 Verilog/SystemVerilog 建模，核心 RTL 目录按功能拆分：

- YH_rv_cpu.v: 流水线顶层、控制流、冒险控制、级间寄存器；
- YH_rv_cpu_id_stage.v 与 YH_rv_cpu_decoder.v: 译码与控制信号生成；
- YH_rv_cpu_ex_stage.v 与 YH_rv_cpu_alu.v: 执行、乘法和位操作；
- YH_rv_cpu_mem_stage.v: 访存宽度、写掩码和非对齐检查；
- YH_rv_cpu_soc.v: ROM、RAM、MMIO 和仿真/FPGA 共用 SoC。

3.2 编码约定

1. 参数化开关集中在顶层和 SoC 边界传递，例如 ENABLE_ZMMUL_EXTENSION、ENABLE_BITMANIP_EXTENSION、ENABLE_ID_BRANCH_EX_FORWARD。
2. 时序逻辑使用 always @(posedge clk or negedge rst_n)，组合逻辑使用 always @* 或连续赋值，避免隐式 latch。
3. 复位值在级间寄存器中显式给出，flush、stall 与 trap 路径优先级在 CPU 顶层集中处理。
4. 新增指令优先通过 decoder、ALU opcode 和定向 TestBench 三处闭环，避免只改执行单元而遗漏非法指令路径。
5. FPGA 可选功能必须能通过参数关闭，以便在 LUT 预算与性能之间快速切换。

3.3 注释与可追溯性

源代码注释重点说明参数用途、控制流优先级、异常处理路径、扩展指令译码以及 Test-Bench 判定条件。提交源码包保留构建脚本、仿真脚本、测试程序和摘要日志，便于评委从文档指标追溯到原始工程证据。

4 验证计划与测试报告

4.1 验证策略

验证分为四层：指令与模块定向测试、SoC smoke 测试、性能基准测试、FPGA 综合实现与下载测试。文中所有性能与实现指标均来自脚本化构建和日志解析，不依赖人工填报数据。

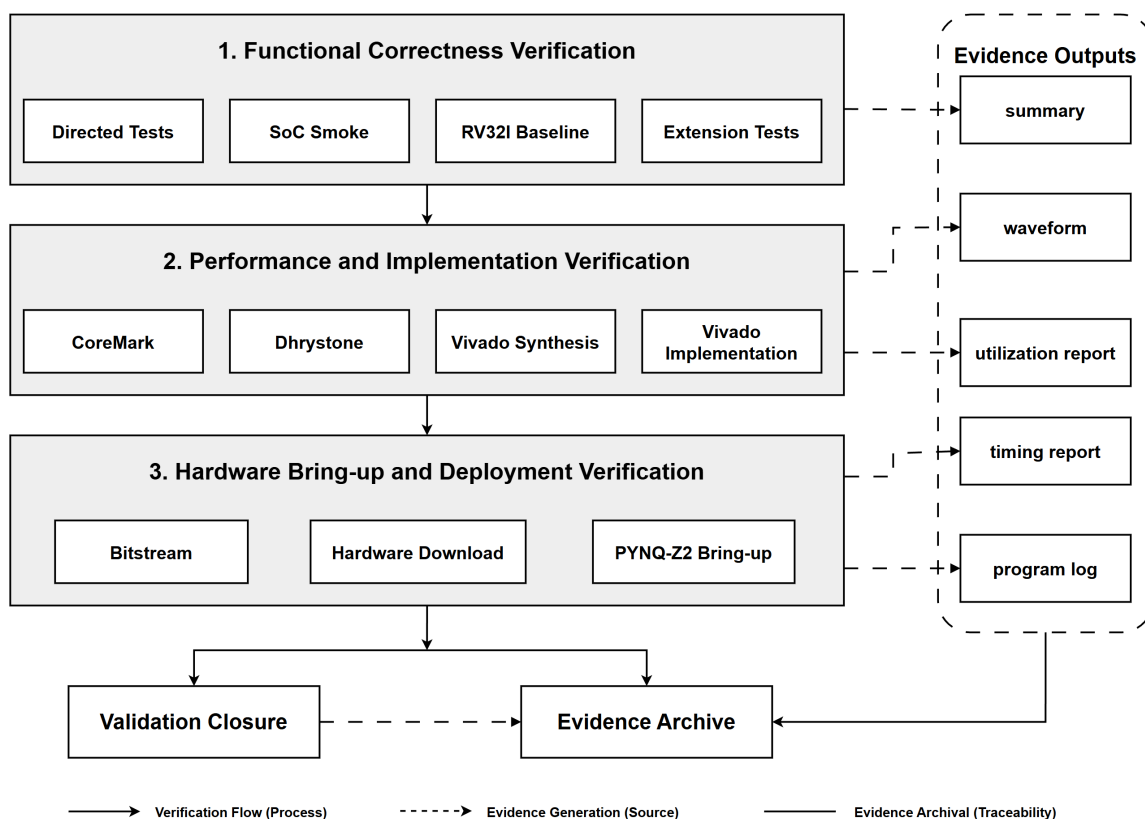


图 3 验证闭环与证据流图

表 3 回归测试矩阵

测试项	覆盖内容	结果
SoC smoke	启动、ROM/RAM、UART 输出、基本指令流。	PASS
Zmmul 诊断	mul 正确性与 divu 非支持异常路径。	PASS
Bitmanip 提交路径	Zba、Zbb、Zbs 快速子集和非支持 Zbc 拒绝路径。	PASS
探索扩展路径	Zbc、XThead condmov/ext/addsl、indexed memidx 与 IDBR 参数化回归。最终冻结 bitstream 未启用。	PASS/未启用
Branch predict diag	静态 backward BNE 预测与控制流回退。	PASS
CoreMark	10 iterations, 2K performance profile, CRC 匹配。	4.137461 Core-Mark/MHz
Dhrystone	Dhrystone 2.2, 运行 10 次, 采用优化构建并复验结果。	2.908287 DMIPS/MHz
PYNQ-Z2 实现	综合、布局布线、bitstream、硬件下载。	PASS

4.2 关键性能结果

表 4 性能测试结果

指标	结果
CoreMark/MHz	4.137461
CoreMark ticks	2416941
Iter/s @ 100MHz	413.746136
Dhrystone DMIPS-MHz	2.908287
指令覆盖说明	覆盖 RV32I 基础指令、Zmmul、Zba/Zbb/Zbs 及非法/非支持指令路径；Zbc、XThead、IDBR 探索路径保留定向回归记录。
分支预测说明	定向诊断 PASS；Profile 显示 JAL EX redirect 降为 0，相关错误事件未检出。

4.3 优化前后差异

以未采用本作品控制流优化组合的 RV32IM 参考路径 4.060276 CoreMark/MHz 为对照，最终冻结路径达到 4.137461 CoreMark/MHz，并在 Dhrystone 优化构建复验中达到 2.908287 DMIPS/MHz。性能提升主要来自 Zmmul 与 Bitmanip 热点扩展、低面积前端控制流优化和可关闭参数化组织；与此同时，设计保持 4934 LUT 的资源占用与正裕量时序结果，满足 LUT < 5000 的板级约束。高性能探索路径曾达到更高 CoreMark，但因 6147 LUT 与

WNS=-0.801 ns 未进入冻结提交口径。

5 FPGA 适配指南

5.1 板级配置

表 5 PYNQ-Z2 适配配置

项目	配置
开发板	Xilinx PYNQ-Z2
器件	xc7z020clg400-1
输入时钟	PYNQ-Z2 板载 125 MHz PL 时钟
CPU 时钟	MMCM 生成 50.0 MHz，满足不低于 50 MHz 要求
约束文件	fpga_artifacts_pynq_z2/constraints/pynq_z2_template.xdc
bitstream	PYNQ-Z2 提交 bitstream，完整文件名见证据索引。
下载证据	Vivado Hardware Manager 日志，含 PROGRAM_OK: xc7z020_1。

5.2 资源与时序

表 6 实现后资源与时序结果

指标	结果
Slice LUTs	4934 / 53200 = 9.27%
Slice Registers	2327 / 106400 = 2.19%
Block RAM Tile	4 / 140 = 2.86%
DSP	15 / 220 = 6.82%
Setup WNS	+0.440 ns，0 个失败端点
Hold WHS	+0.151 ns，0 个失败端点
硬件下载	Vivado Hardware Manager 检出 xc7z020_1 并完成 PROGRAM_OK

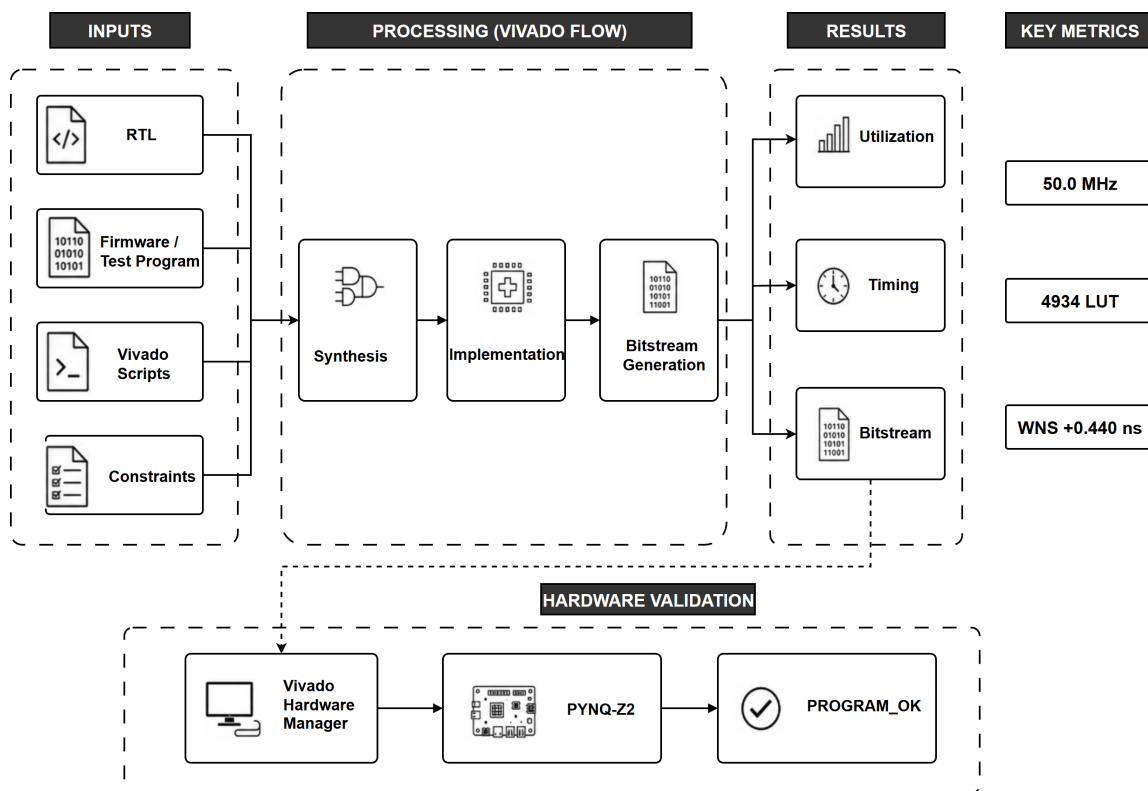


图 4 FPGA 原型适配路径图

5.3 上板连接说明

JTAG 下载和器件识别通过 PYNQ-Z2 Micro-USB 完成。软核 UART 约束到 Pmod B，若需采集串口日志，应使用外接 3.3 V USB-UART 模块连接：

- uart_rxd_out: FPGA TX, Pmod B1 / Y14;
- uart_txd_in: FPGA RX, Pmod B0 / W14;
- 电平: 3.3 V。

板级验证证据以 bitstream 下载日志、器件识别、实现报告、仿真回归结果和基准测试摘要为主。PYNQ-Z2 的 Micro-USB 串口连接至 PS 端，不直接连接 PL 软核 UART；软核 UART 已约束到 Pmod B，可通过外接 3.3 V USB-UART 采集文本日志。未接入外部 USB-UART 时，材料以 JTAG 下载成功、资源时序报告和脚本化验证日志作为板级证据。

6 AI 工具使用声明

本项目在非核心创作环节使用 AI 工具辅助研发与材料整理，工具包括 OpenAI 与 DeepSeek。AI 未替代核心 CPU 架构决策、RTL 最终确认和实验结论判定。

表 7 AI 工具使用情况

工具	使用场景	使用比例估计
OpenAI	工程脚本梳理、CoreMark 热点分析、优化方案筛选、回归命令组织、日志快速核对。	文档约 20–30%；代码修改建议低于 5%
DeepSeek	技术文稿润色、表述压缩、章节逻辑检查。	文档润色约 10–20%

7 结论

YH_rv_cpu 形成了完整的初赛设计证据链：CPU 架构与 RTL 建模边界明确，功能验证与性能测试可追溯，PYNQ-Z2 下载成功，资源占用与时序结果满足板级约束。该设计在保持轻量级实现和低资源占用的同时，通过热点扩展保留、前端控制流优化和参数化工程组织取得了 4.137461 CoreMark/MHz 与 2.908287 DMIPS/MHz 的实测表现，体现了在 FPGA 资源受限条件下对性能、面积与工程可实现性的综合平衡。

参考文献

- [1] RISC-V International. The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture. <https://riscv.github.io/riscv-isa-manual/snapshot/unprivileged/>
- [2] RISC-V International. Bit-Manipulation Extension. <https://docs.riscv.org/reference/isa/unpriv/b-st-ext.html>
- [3] K. Asanovic and D. A. Patterson. Instruction Sets Should Be Free: The Case For RISC-V. EECS Technical Report, 2014. <https://people.eecs.berkeley.edu/~krste/papers/EECS-2014-146.pdf>
- [4] T. Miyazaki et al. RVCoreP: An Optimized RISC-V Soft Processor of Five-Stage Pipelining. IEICE Transactions on Information and Systems, 2020. <https://arxiv.org/abs/2002.03568>
- [5] T. Miyazaki et al. RVCoreP-32IM: An Effective Architecture to Implement Mul/Div Instructions for Five Stage RISC-V Soft Processors. 2020. <https://arxiv.org/abs/2010.16171>
- [6] F. Zaruba et al. The Cost of Application-Class Processing: Energy and Performance Analysis of a Linux-ready 1.7GHz 64-bit RISC-V Core in 22nm FD-SOI Technology. 2019. <https://arxiv.org/abs/1904.05442>

证据索引

证据项	路径
CoreMark 摘要	性能与验证报告/性能日志/ 中的提交路径 CoreMark 摘要。
Dhrystone 摘要	性能与验证报告/性能日志/ 中的 Dhrystone 摘要。
实现资源报告	FPGA 原型系统/fpga_artifacts_pynq_z2/reports/impl_utilization.rpt
实现时序报告	FPGA 原型系统/fpga_artifacts_pynq_z2/reports/impl_timing_summary.rpt
硬件下载日志	FPGA 原型系统/fpga_artifacts_pynq_z2/logs/ 中的 PYNQ-Z2 下载日志。
bitstream	FPGA 原型系统/fpga_artifacts_pynq_z2/ 中的 PYNQ-Z2 bitstream。